Do Deterministic Verifiers Reduce Unsafe LLM-Generated Network Changes? A Paired Shared-Candidate Evaluation

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired experiment (autocand_A1_prereg_v1) that measured whether coupling a deterministic NetOps verifier+gate (generate→validate→repair, max 1 automated repair) to a single LLM (gpt-5-mini) changes safety outcomes when the identical initial LLM candidate is observed with and without gating. Using shared_initial_candidate semantics, one sampled generation was reused for both arms per intent. We evaluated 200 deterministically selected paired intents in a sandboxed Batfish-style validator (deterministic_netops_validator_v5). All 200 shared initial candidates passed validation in both baseline and guarded arms (contingency n11=200, n10=0, n01=0, n00=0). The preregistered paired estimand (guarded minus baseline unsafe-proposal rate) = 0.0; exact two-sided p = 1.0; 95% paired-bootstrap percentile CI = [0.0,0.0] (bootstrap_seed=1729, resamples=10000). No automated repairs were applied. Under shared_initial_candidate semantics the confirmatory contrast tests only validator/repair/gate effects on identical first-pass candidates; it does not measure guarded-conditioned regeneration, prompt engineering, or multi-sample/model variance. We present the preregistered design, implementation and execution accounting, artifact-grounded interpretation of the null result, and concrete recommendations for follow-up designs that avoid ceiling effects.**

I. Introduction

Generative language models are increasingly proposed to translate high-level operator intent into network device configurations and maintenance workflows. Operational deployment requires evidence that model outputs will not introduce reachability, isolation, or policy violations when applied to live networks. Prior surveys and empirical reports emphasize that operational trustworthiness depends strongly on surrounding system machinery—retrieval, deterministic validators, gates, and repair loops—rather than the model alone [1]–[3]. Architectures that combine generation with deterministic validation and bounded repair (generate→validate→repair) are widely advocated, but reproducible paired evidence that isolates the verifier/gate effect on identical model candidates is limited [3],[4].

Research question. For an intent→configuration task bank and a single LLM, does coupling a deterministic validator and deterministic accept/reject gate (with at most one automated repair attempt) to the identical sampled initial candidate reduce the incidence of unsafe proposals relative to observing that same candidate without gating? We formalized this question as the preregistered estimand paired_success_rate_difference_guarded_minus_baseline under shared_initial_candidate semantics.

Contributions. (1) A fully preregistered paired design and frozen execution (autocand_A1_prereg_v1) that fixed generation semantics, sampling, model, validator, gating policy, and analysis prior to observation; (2) an autonomous confirmatory run using gpt-5-mini and a Batfish-style deterministic validator on n=200 deterministically selected paired intents with shared_initial_candidate semantics; (3) artifact-grounded analysis showing complete concordance (all candidates valid) and no repair activity; (4) an interpretation of ceiling effects grounded in archived artifacts; and (5) concrete, artifact-supported recommendations for follow-up preregistered designs that permit nondegenerate inference of verifier/gate value.

II. Related Work

LLM-assisted NetOps and AIOps research spans intent-based configuration translation, runbook generation, log/RCA, and early agentic automation proposals. Several recent surveys and empirical studies document hallucination and unsafe command risks in operational contexts and argue for validator-coupled pipelines, RAG anchoring, and external safety checks [1],[2],[5]. In adjacent domains, generate–validate–repair pipelines have been shown to improve safety or repairability for program repair and DevOps tasks under specific datasets and validators [6],[7].

Methodological gaps. Two gaps motivate our paired design. First, pipeline comparisons that allow different first-pass model draws per arm confound validator effects with model sampling variance. Second, many existing benchmarks emphasize QA/relevance metrics or log/RCA tasks but lack tightly coupled intent→config items evaluated under deterministic validators with paired protocols that preserve initial candidate sampling. Our shared_initial_candidate experiment isolates the validator/gate effect on identical initial generations and therefore directly addresses the first gap while demonstrating the practical implications of generation semantics for inference.

III. Methodology

Preregistration and frozen plan. The study was preregistered as autocand_A1_prereg_v1. The preregistration fixed: the primary estimand (paired_success_rate_difference_guarded_minus_baseline);

generation semantics = shared_initial_candidate; model/adaptor (gpt-5-mini via hosted_netops_gvr_v1); deterministic validator (deterministic_netops_validator_v5) and gate policy; repair budget (maximum_repair_calls_per_task = 1); task selection rule and sample size (task_count = 200 paired intents); and the confirmatory analysis executor (paired_binary_analysis_v1). These items and parameter values were frozen before observing outputs and are archived in the archived model configuration and the preregistration manifest (preregistration_sha256 recorded in the execution manifest).

Generation semantics clarification (preregistered and executed). The executed and preregistered generation semantics were shared_initial_candidate: exactly one initial LLM generation was produced and deterministically reused for both baseline and guarded arms per paired intent; therefore the confirmatory estimand evaluates only validator/repair/gate effects on that same first-pass candidate and does not measure any effect of guarded-conditioned regeneration or model sampling variance. This design choice is central to the statistical meaning of the paired contrast.

Models, adapter, validator, and configuration. Declared model: gpt-5-mini (the archived model configuration records declared_model_version="gpt-5-mini" and provider="openai_responses"). Adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5 implementing Batfish-style checks (sequence ordering, interface precondition checks, final_group and minimum_up constraints, protected interfaces, and reachability/policy consistency). Model configuration recorded: max_output_tokens=2000, maximum_attempts_per_call=1, reasoning_effort="minimal", and temperature=null (the archived model configuration).

Task bank and deterministic sampling. The task generator deterministic_netops_task_generator_v5 produced intent→configuration items with embedded constraints: allowed_fields, allowed_touched_fields, preserve_unspecified_state, workflow_policies (final_group_constraints and minimum_active_in_groups), and explicit required_sequence lists. The preregistered sampler selected 200 paired intents using the adapter's deterministic index rule so selection is fixed and outcome-independent. Manifest entries include per-task difficulty metadata (labels such as "very_hard" and "extreme"); the manifest indicates the bank is dominated by 'very_hard' items with a minority labeled 'extreme' (the archived the archived task manifest contains exact per-label counts).

Interventions, gating, and scoring. Baseline: inspect the shared initial candidate (no validator gating applied). Guarded: feed the identical shared initial candidate to deterministic_netops_validator_v5; the deterministic gate followed a preregistered accept/reject rule and, upon violations, allowed up to one automated repair attempt (maximum_repair_calls_per_task = 1). Primary outcome: binary unsafe_flag = any reachability or workflow/policy violation reported by the validator after candidate stabilization; scoring used rule full_intent_constraint_validation and returned score_reason_code values such as VALID_CONFIGURATION or specific violation codes. Per-episode validator traces (validation_before/validation_after), repair flags, and scoring outputs were archived under the archived execution artifact

Confirmatory analysis (frozen). The primary confirmatory analysis used paired_binary_analysis_v1. The 2×2 paired contingency compares unsafe_flag between baseline and guarded across pairs. The analysis reports discordant pair counts n10 (guarded_only) and n01 (baseline_only), an exact two-sided McNemar-equivalent p-value, and a paired-bootstrap percentile 95% CI (bootstrap_seed=1729, resamples=10,000). Missingness policy (complete_pair_primary) and a one-retry rule for infrastructure errors were preregistered; the run produced no retriable infrastructure failures. The analysis executor and bootstrap parameters are archived in the archived analysis artifact

Implementation and archival hygiene. All model provider traces, shared_initial generations, per-episode responses, validator traces, repair traces (null when unused), task_manifest.jsonl, and analysis outputs (the archived analysis results, the archived paired contingency table) were recorded during execution. Key artifact paths include the archived execution artifact, the archived task manifest, the archived model configuration, the archived execution artifact, the archived execution artifact, and the archived execution artifact Deterministic reconciliation and provider call audits are under the archived analysis artifact

IV. Results

Execution accounting and manifest summary. The run executed as preregistered: started_at_utc = 2026-08-31T13:49:34.191203+00:00 and completed_at_utc = 2026-08-31T14:14:32.660268+00:00 (execution manifest). Planned_episode_count = 400; completed_episode_count = 400; failed_episode_count = 0. Model_calls_used = 200 (one shared initial generation per pair), and hosted_model_call_event_count = 200 in the provider call audit. Cache_hit_count = 0 and cache_miss_count = 200. Stage accounting shows shared_initial_generation = 200 and a single shared condition per pair (the archived analysis artifact).

Primary confirmatory outcome (artifact-grounded). The paired contingency (guarded × baseline) is n11=200, n10=0, n01=0, n00=0 (the archived paired contingency table). Thus every shared initial candidate was scored VALID_CONFIGURATION by deterministic_netops_validator_v5 in both arms. The paired estimate (guarded minus baseline unsafe-proposal rate) = 0.0. The exact two-sided test returned p = 1.0. The paired-bootstrap percentile 95% CI is [0.0,0.0] using bootstrap_seed = 1729 and 10,000 resamples (the archived analysis artifact). The zero-width CI and p = 1.0 are direct arithmetic consequences of complete concordance (no discordant pairs) under the

registered estimand; these artifacts reproduce exactly in the archived analysis outputs.

Repair activity and validator diagnostics. repair_applied = false for every guarded episode; repair_provider_trace_path fields are null in the scoring artifacts (the archived execution artifact). Representative episodes (task-000004 through task-000008) show parsed_command_count, required_command_count, observed_touched_field_count, validator final_state snapshots, and violation_count = 0 for all completed episodes. Example: task-000004 parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and validation_after.valid = true (the archived execution artifact).

Provider audit and reproducibility metrics. Provider audit recorded hosted_model_call_event_count = 200 and completed_hosted_model_call_event_count = 200; cross_condition_cache_key_reuse_observed = false. Deterministic reconciliation asserts episode_accounting_consistent = true and all deterministic consistency checks passed (the archived analysis artifact). Analysis artifacts include the archived condition summary (baseline successes=200, guarded successes=200) and the archived paired contingency table; their SHA-256 manifests are archived in the execution manifest and analysis artifact hashes.

Latency and operational notes. Because shared_initial_candidate semantics required a single model call per pair, model_call accounting equals 200. The guarded pipeline incurred validator execution time but no repair cycles. Timing logs in the archived execution artifact and the archived analysis artifact show median additional per-pair wall time was dominated by deterministic validator evaluation; exact latency distributions are archived for reproducibility. No provider or infrastructure retries were triggered under the preregistered retry policy.

Missingness, contamination, and reconciliation. Missingness_summary shows complete_pairs = 200 with zero failed or unscorable episodes. Contamination_summary.csv is empty (no flagged contamination). Deterministic reconciliation confirms pair accounting: planned_episode_count = 400, completed_episode_count = 400, and complete_pair_count = 200 (the archived analysis artifact).

Representative task patterns. Tasks exercise workflow sequencing constraints (VLAN migration, uplink failover, MTU upgrades), protected interfaces (mgmt1), final_group constraints (exactly one active interface out of a redundancy group), and minimum_up constraints. The archived task_manifest.jsonl encodes initial_state, required_sequence, and workflow_policies; representative entry task-000004 requires the sequence: edge2 down $\rightarrow$ edge2 VLAN 40 $\rightarrow$ edge2 up $\rightarrow$ edge1 down and the validator normalized_configuration matches the reference answer (the archived execution artifact). These artifacts demonstrate validator coverage of sequencing and precondition rules used to define the unsafe_flag.

V. Discussion

Artifact-grounded interpretation of the degenerate null. Under preregistered shared_initial_candidate semantics the confirmatory estimand isolates only validator/repair/gate effects on identical first-pass candidates. Observing n11 = 200 (every shared initial candidate valid) implies that, for the chosen model (gpt-5-mini), the selected synthetic task bank, the prompt/template family used, and the deterministic validator's ruleset, the sampled initial candidates already satisfied the validator constraints; therefore the guarded machinery applied no repairs and changed no acceptance decisions. The archived artifacts (the archived paired contingency table and the archived execution artifact) support this interpretation.

Why the paired bootstrap CI is zero-width. The paired bootstrap CI is degenerate because there are no discordant pairs to resample; every bootstrap resample of paired outcomes yields the same paired difference (0.0). This produces zero empirical variance and therefore a CI [0.0,0.0]. This behavior is recorded in the archived analysis artifact (bootstrap_seed = 1729; resamples = 10000) and is a legitimate inferential outcome rather than an analysis error.

Ceiling-effect contributors supported by artifacts. The execution artifacts point to three concrete contributors: (1) task-bank alignment—deterministic_netops_task_generator_v5 synthesized intents with required_sequences closely matching the expected prompt schema, increasing the probability of syntactically correct model outputs (representative task payloads show required_sequence lists and workflow_policies); (2) validator tightness—the deterministic_netops_validator_v5 rule set encoded the same sequencing and precondition checks used to synthesize reference answers so a correct syntactic mapping passed validation readily (validator traces show parsed_command_count == required_command_count for representative episodes); and (3) shared_initial_candidate semantics prevented any guarded-conditioned regeneration or model variance that might have produced repairs or discordant outcomes. Each of these contributors is traceable in the archived task_manifest.jsonl, the archived execution artifact, and the archived model configuration.

Design recommendations grounded in execution artifacts. To measure verifier/gate value under nondegenerate conditions we recommend: (a) augmenting the task bank with adversarial or out-of-distribution intents to raise baseline unsafe rates—the archived task_manifest.jsonl contains generation_seed fields and difficulty labels that researchers can modify to inject adversarial seeds; (b) executing an alternative confirmatory design that uses independent generation per condition (two independent model calls per pair) or multiple independent samples per condition; this change transforms the estimand to capture guarded-conditioned regeneration and model sampling variance and increases model_call accounting (up to two model calls per pair, i.e., up to 400 model calls for the 200 pairs under a two-call regime); (c) testing multiple model variants or sampling temperatures to reveal model variance effects

(the archived model configuration records requested_model and temperature fields and can be adapted); (d) increasing the automated repair budget or enabling iterative repair loops to evaluate repair effectiveness when initial violations are nontrivial; and (e) including human-in-the-loop arms to assess operator decisions in response to validator flags.

Operational implications and caveats. The null result does not imply that deterministic validators lack value in more realistic or adversarial regimes; rather it demonstrates that design choices (task bank, generation semantics, validator rules) determine whether a paired shared-candidate contrast can detect verifier value. Our artifacts show that, for this run, the model and tasks produced validated configurations reproducibly. Field deployments would need additional evaluation regimes (adversarial item injection, multiple model draws, and HITL arms) to quantify operational benefit and risk.

Reproducibility and artifact-driven follow-ups. All per-episode responses, provider traces, validator traces, and analysis artifacts are archived (the archived execution artifact, the archived task manifest, the archived model configuration, the archived analysis results, the archived paired contingency table). Researchers can replay the exact run under the same adapter to reproduce the confirmatory null result; they can also modify the manifest to insert adversarial seeds or change generation semantics to independent per-condition sampling and rerun. The execution manifest contains representative SHA-256 artifact hashes and paths (see execution_manifest representative_artifact_hashes) that support byte-level verification of reproduced runs.

VI. Limitations

- Internal validity: shared_initial_candidate semantics mean baseline and guarded conditions began from an identical initial candidate; the confirmatory estimand therefore measures only validator/repair/gate effects on that same candidate and cannot assess effects of guarded-conditioned regeneration or prompt engineering.
- Construct validity: tasks were synthetic and deterministically selected; the binary unsafe_flag depends on validator coverage (reachability and workflow sequencing rules) and may not capture all operational risks (transient performance regressions, vendor-specific side effects outside validator rules).
- External validity: single LLM (gpt-5-mini), single deterministic validator, and synthetic intents limit generalizability to other models, validators, production topologies, or live device interactions.
- Conclusion validity: the observed null effect (paired difference = 0) reflects this execution's sampling frame where baseline generation produced only valid candidates; the study was powered to detect ~10 percentage-point reductions but cannot rule out smaller effects under different sampling or adversarial inputs.
- Operational relevance: no live device changes were executed (sandboxed validation only); operational benefits (reduced triage time, deployment safety) were not measured.
- Security/contamination: adversarial or poisoning scenarios were not exercised; no retrieval-augmented generation (RAG) was used, so retrieval-based poisoning vectors were not evaluated.
- Task-bank reporting: the archived task_manifest.jsonl is authoritative for per-label counts; manifest metadata indicates tasks are dominated by 'very_hard' items with a minority labeled 'extreme' (exact per-label counts are available in the archived manifest).

VII. Conclusion

We executed a fully preregistered paired experiment under shared_initial_candidate semantics comparing an LLM-alone observation to the same initial candidate after deterministic verification and bounded automated repair. Every sampled initial candidate satisfied the deterministic validator and the guarded pipeline applied no repairs; the paired difference = 0.0 (p = 1.0) with a paired-bootstrap 95% CI = [0.0,0.0]. This null result is specific to the chosen model (gpt-5-mini), the synthetic task bank, the deterministic selection, the validator ruleset, and the shared_initial_candidate semantics; it does not contradict prior evidence that validators can reduce unsafe outputs under different experimental regimes (e.g., independent condition generation, adversarial inputs, multiple model samples) [1]–[3],[6]. We archive full artifacts and recommend follow-up preregistered designs that intentionally increase baseline unsafe rates or permit independent per-condition generation to quantify verifier/gate contributions under more challenging conditions. All core artifacts and analysis outputs supporting these conclusions are archived and referenced in this manuscript's evidence bundle.

References

[1] N. Tu, S. Nam, and J. W.-K. Hong, "Intent-Based Network Configuration Using Large Language Models," Network and Service Management, 2024. 10.1002/nem.2313.

[2] M. Bilal, J. Crowcroft, R. Wang, X. Xu, and S. Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety," 2026. arXiv:2605.12729.

[3] B. Zhang et al., "LLM-Style DevOps Copilot for Cloud-Native Troubleshooting: Retrieval-Augmented Runbook Generation and Command-Safety Evaluation," J. Tie, 2026. 10.51903/jtie.v5i2.534.

[4] S. Chakraborty, N. Chitta, and R. Sundaresan, "Automation of Network Configuration Generation using Large Language Models," in Proc. CNSM 2024, 2024. 10.23919/cnsm62983.2024.10814407.

[5] T. Cui et al., "LogEval: A Comprehensive Benchmark Suite for Large Language Models In Log Analysis," 2024. arXiv:2407.01896.

[6] F. Abu Zaid, D. Neider, and M. Yalçıner, "VeriFlow: Modeling Distributions for Neural Network Verification," AAAI 2026 findings, 2026. 10.1609/aaai.v40i33.40030.

Acknowledgements: None declared.

Disclosure Statement

Preregistration and autonomy: this study was preregistered as autocand_A1_prereg_v1 and executed autonomously after the autonomy lock. Models and tools: single LLM = gpt-5-mini (provider recorded as openai_responses); orchestration/adapter = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5; task generator = deterministic_netops_task_generator_v5. Human role and autonomy boundary: the human Research Director selected the topical family and constrained the initial prompt pre-lock; all literature review, preregistration, code generation, experiment execution, analysis, peer-review emulation, and manuscript drafting occurred autonomously after the lock; no human manually edited manuscript technical content after the lock. Generation semantics clarification: the executed and preregistered generation semantics were shared_initial_candidate (one initial sampled generation reused by both arms per pair); therefore the confirmatory estimand measures only validator/repair/gate effects on identical first-pass candidates rather than effects from guarded-conditioned regeneration or multiple independent model samples. Archived immutable master prompt: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba). Key archived artifacts: the archived execution artifact, the archived task manifest, the archived model configuration, the archived analysis results, the archived paired contingency table, and the full execution_manifest. The preregistration id is autocand_A1_prereg_v1.

References

[1] https://doi.org/10.1002/nem.2313
[2] https://doi.org/10.1609/aaai.v40i33.40030
[3] https://arxiv.org/abs/2605.12729
[4] https://doi.org/10.1038/s41591-024-03445-1
[5] https://doi.org/10.23919/cnsm62983.2024.10814407
[6] https://doi.org/10.48550/arXiv.2407.01896